

Cryptographic Validity Is Not Runtime Admissibility

Opaque Reasoning Traces, Dynamic Classification Failure, and a Synthetic Policy-Conformance Benchmark

Ignacio Adrián Lerer

Attorney (UBA), Executive MBA (IAE, Universidad Austral).

Buenos Aires, Argentina.

adrian@lerer.com.ar

ORCID 0009-0007-6378-9749

August 2026

Abstract

Recent work showed that encrypted reasoning traces returned by major large-language-model APIs could be replayed across sessions, users, and compatible models, allowing a weaker model to expose hidden reasoning, recover sensitive information, or execute an instruction invisible in the public transcript. The disclosed attacks were subsequently mitigated, but the episode reveals a broader architectural error. Cryptographic authenticity answers whether an artifact was produced and preserved under an accepted mechanism. It does not answer whether the artifact remains appropriate to trust in a different context. This paper develops a theory of contextual runtime admissibility for agentic memory. It distinguishes cryptographic, contextual, semantic, and operational validity and formalizes a Contextual Runtime Admissibility Gate, or CRAG. A deterministic benchmark compares syntax-only acceptance, provenance validation, context binding, and full CRAG across 28 synthetic cases, half reserved as a combinatorial holdout. Context binding alone admitted 60% of unsafe cases overall and 70% on holdout. Full CRAG admitted none while preserving all eight benign-continuity cases. These results establish conformance to a frozen normative contract, not independent construct validation or production effectiveness. The framework interprets unsafe replay as a Dynamic Classification Failure and offers a bounded Extended Phenotype Theory account of traces that persist outside the process that produced them without yet qualifying as evolutionary replicators. Integrity must be verified, but no persistent agentic artifact should acquire authority to influence action merely because it is authentic.

Keywords: agentic memory; reasoning traces; runtime admissibility; Dynamic Classification Failure; prompt injection; memory poisoning; extended phenotype; AI governance

1. Introduction

A signed artifact carries a powerful presumption. The signature tells a system that the artifact is genuine, unmodified, and suitable for the verification procedure that produced it. In ordinary software, those properties often support a further decision: the system accepts the object and proceeds. Agentic systems make that inference dangerous. The object is not merely data. It may become context, memory, instruction, evidence, a premise for planning, or a cause of tool use.

Panfilov et al. (2026) expose this distinction through an unusually clear case. Several leading AI providers returned hidden reasoning to clients in opaque or encrypted blocks so the blocks could be supplied again during later API calls. The design preserved statelessness and conversational continuity. Yet, during the authors' testing period in early July 2026, the blocks were sufficiently portable across sessions, users, and compatible models that a weaker model could process a trace generated by a stronger model and be induced to transcribe it. The infrastructure did not surrender its cryptographic keys. It did

something more revealing: it performed the authorized decoding operation in a context where authorization to decode had been mistaken for authorization to disclose, trust, or act.

The reported consequences were not confined to intellectual property. The authors reconstructed 315,320 reasoning blocks collected from 6,708 publicly available agent trajectories. They found sensitive artifacts in a subset of those traces, including credentials and personally identifiable information. They also demonstrated an invisible prompt injection: a signed reasoning block containing a malicious instruction influenced a later model to upload information even though the instruction was absent from the visible conversation. The attacks described in the paper ceased to be reproducible after responsible disclosure and provider mitigation. That fact narrows the present claim. It does not eliminate it.

The episode should not be reduced to an encryption failure. The encryption performed its narrower task: it concealed content from the client and detected tampering. The system failed at a different boundary. It treated the artifact's technical validity as sufficient authority for reuse. This is an instance of a general problem in agent architecture. A memory can be authentic and irrelevant. A retrieved record can be genuine and stale. A signed instruction can be validly issued and no longer authorized. A summary can accurately reflect an earlier state while becoming unsafe under expanded tool permissions. Provenance is necessary. Provenance alone does not establish admissibility.

This paper asks: under what conditions may an AI agent rely on and act upon an opaque memory artifact that is cryptographically authentic but has moved beyond its original context? It answers by separating four validity predicates: cryptographic validity, contextual validity, semantic validity, and operational validity. These predicates are related but not interchangeable. The paper then proposes a Contextual Runtime Admissibility Gate, or CRAG, that evaluates the artifact at the moment of proposed reliance rather than only at creation or transport.

The argument extends the distinction between legitimacy formation and legitimacy execution developed in Runtime Admissibility Before Reliance (Lerer, 2026a). It makes four contributions. First, it reframes encrypted-trace replay as an authority problem, not only a confidentiality problem. Second, it applies Dynamic Classification Failure to agentic memory: the artifact remains unchanged while its operative classification changes because the user, model, purpose, permissions, or environment has changed. Third, it provides a bounded evolutionary interpretation. A persistent trace can function as an extended phenotype by modifying later behavior outside the process that produced it. Persistence and causal efficacy, however, do not make the trace a replicator. Fourth, the paper converts the conceptual distinction into a falsifiable research program and a system gate with explicit denial, quarantine, inspection, and escalation states.

The argument is deliberately narrower than a claim that hidden reasoning should always be disclosed or always be destroyed. Reasoning transparency creates its own hazards. Nor does the paper claim that the vulnerabilities documented in July 2026 remain active. Its target is the architectural inference that survives any particular patch: if an object passes an authenticity check, the system may still lack sufficient grounds to let that object shape a new decision.

2. The Encrypted-Trace Episode

2.1 A stateless continuity mechanism

Modern reasoning models may generate intermediate computations before returning a user-facing answer. Providers have reasons not to expose the complete trace. Detailed reasoning may reveal proprietary methods, safety behavior, sensitive context, or harmful intermediate content. At the same time, multi-turn

and tool-using workflows need continuity. One architectural response is to return an opaque representation of the reasoning to the client and require the client to pass it back unchanged when the conversation continues.

Official API documentation confirms the functional pattern. Anthropic describes a `signature` field as an encrypted copy of full thinking that may be passed back for multi-turn or tool-use continuity, and states that the field verifies that a thinking block was generated by Claude. OpenAI documentation instructs clients using stateless or zero-data-retention configurations to replay encrypted reasoning items. Google documentation similarly requires thought signatures to preserve reasoning context in relevant function-calling workflows. These interfaces differ in implementation, but they share a structural feature: an opaque artifact crosses the client boundary and later returns as context.

The design distributes storage and permits continuity without requiring the provider to retain every internal state. It also creates an unusual trust arrangement. The client possesses an artifact it cannot inspect. The provider can interpret it. The model may receive it as prior reasoning. The user may publish, archive, copy, sanitize, or import the surrounding transcript without knowing what the opaque component contains.

2.2 What Panfilov et al. demonstrated

Panfilov et al. identified three forms of compatibility: replay within or across sessions, replay across users, and replay across models within a provider ecosystem. They exploited cross-model differences in safeguard strength. A capable model produced the opaque trace; a weaker compatible model received it; an extraction prompt induced the weaker model to expose reconstructed reasoning.

This was not cryptanalysis in the conventional sense. The attacker did not derive a key or defeat the cipher. The server accepted the envelope and supplied its content to a model because the envelope satisfied the provider's technical acceptance conditions. The model became a decryption oracle only because the system combined legitimate decoding with insufficiently bounded use.

The paper presents four attack classes. The first is extraction for potential distillation. Detailed traces may be more valuable than final answers because they include decomposition, discarded paths, heuristics, and intermediate deductions. The second is privacy leakage. Publicly shared trajectories may contain opaque reasoning that repeats or introduces credentials and personal data absent from the sanitized visible transcript. The third is recovery of harmful intermediate information that the visible answer withholds. The fourth is invisible prompt injection, where an instruction embedded in an opaque trace affects subsequent planning or tool use.

The fourth attack is decisive for the present theory. Confidentiality loss can be described as unauthorized disclosure. Invisible injection shows something else: an authentic artifact can become an unauthorized source of practical authority. It crosses from being an object the model can read to an object the agent treats as a reason for action.

2.3 Evidentiary limits

The source paper is strong evidence of the demonstrated architecture and attack paths, but it does not establish every proposition that might be built upon them. The authors lacked the ground-truth plaintext trace needed to verify every reconstructed token. Token-count correspondence, exclusive information, internal coherence, and successful recovery of secrets support high-fidelity extraction; they do not prove perfect verbatim identity in every case.

The public corpus was also a targeted, non-exhaustive sample. The reported trajectory-level leakage rate cannot be generalized to all agent sessions. The exploratory comparison between proprietary traces and outputs from Kimi-K3 or GLM-5.2 does not causally establish that either model was distilled from a named source. The authors say so expressly.

Most important for temporal accuracy, the disclosed procedures stopped working after providers received the report. The paper states that, by August 2026, the Figure 1 attacks were no longer reproducible using the published methods. The correct object of theory is therefore not a presently verified zero-day. It is the class of design error made visible by a historically bounded incident.

3. Four Forms of Validity

Security architectures often compress several questions into a single acceptance decision. Agentic systems need the questions separated because an artifact may pass one and fail another.

Let an artifact be m and the proposed runtime context be c . Context c includes the authenticated principal, session, conversational position, model, task purpose, data classification, available tools, and permission set. Four predicates matter.

3.1 Cryptographic validity

Cryptographic validity, $V_{\text{crypto}}(m)$, asks whether the artifact satisfies the relevant integrity and origin checks. It may establish that the artifact was generated by a recognized system, has not been altered, and conforms to an accepted envelope format.

This property is about the artifact and the verification mechanism. It does not establish that the current caller owns the artifact, that the current task needs it, or that the current model should treat it as prior reasoning. Saltzer and Schroeder's principle of complete mediation is instructive: authority should be checked for every access to every object, not inferred permanently from an earlier decision (Saltzer & Schroeder, 1975).

3.2 Contextual validity

Contextual validity, $V_{\text{context}}(m,c)$, asks whether the artifact belongs in the present relationship. At minimum, the check should address principal, tenant, matter or project, session, predecessor state, conversational order, model compatibility, and temporal validity.

An artifact may be cryptographically valid while contextually invalid. A signed trace copied from another user's trajectory is the simplest case. Less obvious cases include a legitimate memory from the same user but a different confidential matter, a pre-compaction trace restored at the wrong ordinal position, or a trace generated before permissions were materially expanded.

3.3 Semantic validity

Semantic validity, $V_{\text{semantic}}(m,c)$, asks what role the content may legitimately play. Is it evidence, a user preference, a tentative hypothesis, a tool result, a prior model inference, or an instruction? Is it still relevant? Can its meaning be inspected? Does it conflict with higher-authority instructions or current verified facts?

Opacity is critical here. If neither the user nor an independent monitor can inspect an artifact, semantic validation cannot be presumed. The artifact may remain usable for a narrowly bounded continuity function, but it should not silently acquire evidentiary or normative weight. A system that labels every

imported trace as `prior_assistant_reasoning` has already made a semantic decision before evaluating whether that label remains warranted.

3.4 Operational validity

Operational validity, $V_{\text{operational}}(m,c)$, asks whether allowing the artifact to influence the current agent is safe given the actions now available. The same text has different risk when the model can only draft a response and when it can send email, upload files, execute code, transfer funds, alter a legal record, or communicate with other agents.

Operational validity is therefore permission-sensitive and consequence-sensitive. It must consider tool scope, reversibility, human approval, blast radius, external communication, confidentiality, and the possibility of propagation. An artifact may be semantically relevant yet operationally inadmissible unless tools are disabled or a human approves the next action.

3.5 Admissibility is conjunctive, but not always binary

A strict baseline can be expressed as:

$$\text{CRA}(m,c) = V_{\text{crypto}}(m) \wedge V_{\text{context}}(m,c) \wedge V_{\text{semantic}}(m,c) \wedge V_{\text{operational}}(m,c)$$

where CRA denotes contextual runtime admissibility. Under a fail-closed rule, failure of any material predicate prevents direct reliance.

Real systems need more than `allow` and `deny`. Some opaque artifacts cannot be semantically inspected but may still support a confined continuity operation. Others may be converted into a safe summary, reissued under a new context binding, or used only after tool permissions are removed. The gate therefore returns one of five states:

1. `ALLOW`: all required predicates pass for the intended reliance.
2. `CONSTRAIN`: the artifact may be used only with reduced tools, scope, or output authority.
3. `QUARANTINE`: preserve for forensic or continuity purposes but exclude from model context.
4. `ESCALATE`: human or specialist review is required.
5. `DENY`: the artifact must not influence the current run.

These states distinguish memory retention from memory reliance. Deleting every uncertain artifact may destroy legitimate continuity. Trusting every authentic artifact makes continuity a channel for inherited error or attack.

4. Dynamic Classification Failure

4.1 Static artifacts, changing environments

Dynamic Classification Failure occurs when a system assigns a category at one time and continues to rely on that category after changes in the entity or environment have altered the consequences of classification (Lerer, 2026b). The object need not mutate. The relationship between the object and its environment may be enough.

Encrypted reasoning traces provide a clean example. At issuance, a trace may be classified as valid continuation state for user `u`, session `s`, model `q`, purpose `p`, and permission set `P`. After replay, the byte sequence remains identical. Yet one or more coordinates change: `u` becomes `u2`, `s` becomes `s2`, `q` becomes

a weaker compatible model, p changes from solving a problem to transcribing hidden content, or P expands from text generation to network and file access.

Let the relevant context at time t be:

$$c_t = (u_t, s_t, o_t, q_t, p_t, d_t, P_t, T_t)$$

where u is principal, s session, o ordinal position, q model, p purpose, d data classification, P permissions, and T available tools. The artifact m may remain constant while admissibility changes:

$$CRA(m, c_0) = 1, \text{ while } CRA(m, c_1) = 0.$$

A system commits Dynamic Classification Failure when it substitutes the earlier decision for the current one:

$$CRA(m, c_1) := CRA(m, c_0).$$

That substitution is tempting because re-evaluation adds latency, state, and engineering complexity. It also preserves the economic benefit of portability. Yet it converts a local authorization into a transferable authority token.

4.2 Authentication does not carry purpose

The confused-deputy problem offers a useful analogy. A privileged program may possess legitimate authority but apply it on behalf of a requester whose purpose does not justify the action (Hardy, 1988). In the encrypted-trace attack, the provider infrastructure and compatible model jointly occupy the deputy role. They are authorized to process an authentic trace for continuity. The attacker changes the purpose to extraction or injection. The mechanism authenticates the object without adequately binding or re-evaluating the purpose.

The analogy has limits. The model is probabilistic, and the artifact may influence rather than explicitly command. Still, the authority structure is similar: a legitimate capability is exercised under an illegitimate interpretation of who or what the capability serves.

4.3 Static permissibility as a design smell

The broader lesson is that static permissibility should be treated as a design smell whenever an artifact can outlive the environment in which permission was granted. This applies to:

- long-term agent memories;
- compressed conversation summaries;
- shared checkpoints and trajectories;
- retrieved documents and embeddings;
- skill files and tool instructions;
- inter-agent messages;
- signed tool outputs;
- cached approvals and delegated credentials.

Each artifact can remain authentic while becoming stale, cross-tenant, misordered, semantically misclassified, or operationally dangerous. Runtime admissibility is the mechanism that converts these possibilities from caveats into system states.

5. The Contextual Runtime Admissibility Gate

5.1 Design principles

The proposed gate follows five principles.

First, it is placed immediately before reliance. Creation-time checks and ingestion-time checks remain useful, but neither can anticipate every later environment. Complete mediation requires a fresh decision at the point where the artifact would enter model context or authorize action.

Second, the gate separates provenance from authority. Provenance evidence is an input. It is not the conclusion.

Third, the gate is consequence-sensitive. A trace may be admissible for read-only summarization and inadmissible for a tool-using agent. Permissions and reversibility belong in the decision.

Fourth, opacity lowers the default trust state. An uninspectable artifact is not necessarily malicious. It is semantically unverified. The system must either confine its role, obtain a trusted transformation, or block reliance.

Fifth, the gate emits a receipt. Every decision should record the artifact identifier or hash, context coordinates, policy version, state, reasons, constraints, and reviewer when applicable. A later incident must be reconstructible without retaining the sensitive content itself.

5.2 Gate inputs

The minimum gate input is a tuple:

$G = (m_id, provenance, principal, tenant, session, predecessor, model, purpose, sensitivity, permissions, tools, visibility, age, policy)$

`m_id` identifies the artifact without exposing its content. `provenance` records the issuing system and verification result. `principal` and `tenant` prevent cross-user or cross-matter confusion. `session` and `predecessor` establish sequence. `model` determines compatibility and safeguard class. `purpose` states the intended use. `sensitivity` describes the highest plausible data class. `permissions` and `tools` define consequences. `visibility` states whether the content can be independently inspected. `age` supports expiry. `policy` identifies the rules applied.

5.3 Decision rules

The following baseline is intentionally conservative:

1. If cryptographic verification fails, return `DENY`.
2. If `principal`, `tenant`, or matter binding fails, return `DENY` and generate a security event.
3. If `session` or `predecessor` binding fails, return `QUARANTINE` unless an explicit, audited fork or migration procedure applies.
4. If model compatibility is broader than the intended safeguard domain, return `CONSTRAIN` or `DENY`.
5. If the artifact is opaque and would supply instructions, factual premises, secrets, or tool-relevant state, return `ESCALATE` or transform it through an approved server-side process.
6. If the purpose differs materially from the issuance purpose, reclassify from first principles.

7. If high-impact tools are available, require independent validation or human approval before the artifact may influence the action plan.
8. If the artifact is expired, superseded, contradicted, or revoked, return `DENY` or `QUARANTINE` according to retention policy.

This design does not require every deployment to store raw reasoning. It requires the system to preserve enough metadata to decide whether reuse remains justified.

5.4 Context-bound envelopes and residual risk

Panfilov et al. propose binding reasoning envelopes to user identity, session, and predecessor state, limiting cross-model replay, rotating legacy keys, detecting anomalous replays, and training compatible models to resist transcription. These measures address the demonstrated paths. They are consistent with the contextual-validity layer.

They do not fully replace semantic and operational checks. A trace can be correctly bound to the same user and session yet contain a harmful instruction acquired from an untrusted webpage. It can be correctly ordered yet stale. It can be safe while the agent is read-only and dangerous after network access is enabled. Cryptographic context binding narrows who can replay an artifact. Runtime admissibility determines what the artifact may do.

5.5 A reference decision record

A minimal receipt can take the following form:

```
artifact_hash: ...
provenance_state: verified
context_match: principal=pass, tenant=pass, session=pass, order=pass
semantic_state: opaque_unverified
operational_state: high_impact_tools_present
decision: constrain
constraints: no_network, no_external_write, human_review_before_action
policy_version: CRA-0.1
timestamp: ...
```

The receipt does not reveal hidden reasoning. It makes the reliance decision inspectable.

6. Opaque Traces as Extended Phenotypes

6.1 The bounded claim

Extended Phenotype Theory directs attention beyond an originating organism or process to structures in the environment that preserve and express causal effects. This paper develops the AI-facing line of inquiry proposed in Lerer (2026c) but narrows it to a specific artifact and causal pathway. Applied carefully, an opaque reasoning trace has this form. It is produced during a model's inference, leaves the immediate process, persists in storage, and can alter the behavior of another invocation. The source process ends; its artifact remains causally active.

This interpretation adds something that the language of `state` alone misses. State sounds passive. A replayed trace can reorganize the future environment: it changes which premises the model treats as already established, which goals remain salient, which files are modified, which tools are called, and which further traces are generated. In the injection experiment, the artifact did not merely preserve history. It extended the causal reach of an earlier instruction into a later agentic episode.

6.2 Replicator, vehicle, phenotype, and niche

Evolutionary vocabulary becomes imprecise when every persistent information object is called a meme or replicator. The encrypted-trace case permits a cleaner separation.

The informational instruction is a candidate replicator only if it is copied with sufficient fidelity and exhibits differential persistence or reproduction. The opaque signed block is a transmission vehicle or envelope. The downstream changes in plans, files, messages, and tool calls are phenotypic effects. The API, repository practices, checkpoint formats, and agent import conventions form part of the selective niche.

The source paper demonstrates persistence, portability, and causal efficacy. It does not measure self-replication, population frequency, inheritance across generations, or selection. A poisoned trace that instructs an agent to copy itself into later checkpoints would create a candidate replication loop. Even then, an evolutionary claim would require measurement.

6.3 From cascade to selection

Small architectural choices can propagate into large consequences without constituting natural selection. Client-side storage reduces provider state. Portability facilitates model switching. Shared formats reduce friction. Public trajectories support reproducibility. Together, those choices create an environment in which opaque instructions may travel farther than visible ones.

Selection enters only when variants compete and reproduce differentially. Suppose visible and opaque versions of the same instruction are introduced into comparable agent trajectories. If opaque variants survive more reviews, produce more downstream copies, or trigger more actions because they evade inspection, they have higher propagation fitness in that environment. The claim is then testable rather than metaphorical.

This distinction protects EPT from becoming a decorative analogy. The present paper uses EPT to identify externalized causal structure. It reserves EGT and replicator dynamics for a later experiment that actually observes differential propagation.

7. Synthetic Policy-Conformance Benchmark

7.1 Hypotheses

The framework yields six hypotheses. The present benchmark tests the control-logic component of H1, H2, H3, and H6. H4 and H5 require behavioral model or population experiments and remain untested.

H1, provenance insufficiency: a provenance-only control will admit artifacts that a contextual gate correctly rejects after changes in principal, session, model, purpose, or permissions.

H2, permission sensitivity: identical artifacts will produce different admissibility decisions when the agent moves from read-only output to high-impact tool access.

H3, opacity penalty: opaque artifacts will require more restrictive states than inspectable artifacts with equivalent provenance because semantic validation is unavailable.

H4, weakest-compatible-decoder effect: when artifacts are portable across models, family-level extraction risk will correlate with the least resistant compatible decoder rather than the strongest source model.

H5, hidden-fitness advantage: absent a contextual gate, opaque malicious instructions will persist or trigger downstream action more frequently than visible equivalents under review processes limited to plaintext.

H6, contextual-gate benefit: CRAG will reduce unsafe reliance more than provenance-only validation while preserving legitimate continuation through `CONSTRAIN`, audited migration, and reissuance rather than universal deletion.

7.2 Design and experimental units

The benchmark uses 28 synthetic artifact-context pairs. Fourteen development cases instantiate individual control families. Fourteen holdout cases transpose or combine those families without introducing new policy primitives. The split was frozen before execution. No model inference, proprietary trace, external network request, or live exploitation was used.

Each case varies:

- same user versus different user;
- same matter versus different matter;
- same session versus imported session;
- correct versus altered sequence;
- same model class versus weaker compatible model;
- original purpose versus extraction or unrelated task;
- no tools versus file, network, email, or code tools;
- inspectable content versus opaque placeholder;
- current versus expired or superseded policy.

The fixture covers benign continuity, invalid integrity, cross-principal and cross-tenant reuse, unattested and attested migration, sequence failure, model mismatch, purpose drift, expiry, revocation, contradiction, hidden instruction, sensitive opaque state, and high-impact tool availability. Every case records a gold state, an allowed-state set, a rationale, whether automated `CONSTRAIN` is safe, and whether benign continuity should be preserved.

7.3 Baselines

B0 accepts every syntactically valid artifact. B1 requires syntactic and cryptographic validity. B2 adds principal, tenant, session, sequence, migration, and model checks. B3 implements full CRAG by adding purpose, contradiction, expiry, revocation, opacity, sensitivity, instruction, and tool-consequence checks.

The comparison does not treat maximum refusal as success. `ALLOW` and safe `CONSTRAIN` preserve automated continuity; `QUARANTINE`, `ESCALATE`, and `DENY` prevent automated reliance. Unsafe reliance occurs when `ALLOW`, or a case-defined unsafe `CONSTRAIN`, is applied to a non-benign case.

The predeclared acceptance rule required B3, on both splits, to produce zero unsafe automated reliance, at least 90% benign continuity, at least 90% allowed-state accuracy, strict improvement over B2 on unsafe reliance, and no continuity loss relative to B2.

7.4 Results

All five predeclared conditions passed on both splits. Table 1 reports aggregate results.

Policy	Allowed-state accuracy	Unsafe automated reliance	Benign continuity	Automated coverage
B0 syntax only	10.7%	100%	100%	100%
B1 provenance	14.3%	95%	100%	96.4%
B2 context binding	46.4%	60%	100%	71.4%
B3 full CRAG	100%	0%	100%	28.6%

Table 1. Aggregate policy-conformance results across 28 synthetic cases.

On the holdout, B2 automatically admitted 70% of unsafe cases. B3 admitted none. Both policies preserved 100% of benign continuity. B3 returned an allowed state for every holdout case, escalated 28.6%, and used quarantine or denial for the remaining unsafe cases.

The progression identifies what each layer contributes. Syntax and provenance preserve continuity but supply almost no protection against contextually valid-looking misuse. Binding removes cross-principal, cross-tenant, and sequence failures. It does not address a same-context artifact that is expired, revoked, contradicted, repurposed, opaque, sensitive, or newly connected to consequential tools. Semantic and operational classification account for the remaining difference.

7.5 Interpretation and anti-circularity

The perfect B3 score requires restraint. The gold labels and B3 instantiate the same author-defined normative contract. The benchmark therefore tests implementation fidelity and exposes differences among nested policies. It does not independently validate CRAG as a construct, estimate effectiveness against adaptive adversaries, or show that an LLM agent will obey the gate.

The holdout reduces case memorization but does not solve the deeper circularity because it combines known policy dimensions. Stronger validation requires independent expert labeling, policy mutation tests, behavioral trials with tool-using agents, shadow evaluation in real workflows, and measurement of latency, false denial, reviewer burden, and recovery.

7.6 Reproducibility and ethics

All cases, secrets, identities, instructions, and tool consequences are synthetic. The run is deterministic and local. The package preserves the fixture, policy code, unit tests, result JSON, hashes, predeclared acceptance rule, comparison record, and completion status. Network and external-write behavior is represented only by booleans. The benchmark does not replay proprietary reasoning signatures or test patched vendor endpoints.

These constraints are not peripheral. A paper about admissibility would contradict itself if its evidence package created a new unsafe replay channel.

8. Governance Implications

8.1 The user cannot sanitize what the user cannot inspect

Opaque reasoning creates an asymmetric custody problem. The client stores and may publish the artifact; the provider retains the capacity to interpret it; the user cannot verify its contents. Sanitizing the visible

transcript does not certify the opaque field. Consent to share the transcript is therefore incomplete if the user cannot know what is being shared.

A practical rule follows: opaque reasoning fields should be treated as unreviewed sensitive content. They should be excluded from public datasets, shared checkpoints, support bundles, and cross-tenant logs unless a provider supplies a verified sanitization or reissuance procedure. This rule is stricter than ordinary log hygiene because opacity prevents content-level review.

8.2 Provider responsibility and system responsibility

Providers control envelope design, key scope, model compatibility, revocation, and server-side behavior. Developers control storage, logging, publication, import, tool permissions, and whether opaque artifacts enter later context. Neither side can discharge the entire problem onto the other.

Provider controls should bind artifacts to the narrowest practicable identity and context, retire legacy formats, and disclose portability and sanitization properties. Developer controls should strip opaque fields from public records, isolate matters and tenants, re-evaluate imported state, and prevent unverified memory from authorizing high-impact action.

NIST's Generative AI Profile frames risk management across the lifecycle and calls for testing, evaluation, verification, and validation of information-security and provenance risks (Autio et al., 2024). OWASP's agentic guidance identifies memory and context poisoning as a persistent attack surface affecting future reasoning and tool use. Contextual runtime admissibility supplies a specific control point within that broader governance landscape.

8.3 Reliance in legal and institutional systems

The distinction is especially important where AI output may produce institutional consequences. A legal agent may retrieve a valid authority from the wrong jurisdiction, a genuine client instruction from another matter, or an authentic prior analysis superseded by a later ruling. Cryptographic integrity does not cure contextual inadmissibility.

Institutional reliance requires a chain from authority to context to interpretation to permitted consequence. If any link changes, the system must revisit the classification. The same logic applies to compliance approvals, financial instructions, medical records, and administrative decisions. An authentic artifact can be inadmissible evidence, an inadmissible instruction, or an inadmissible basis for action.

This paper does not propose that a model decide legal admissibility autonomously. It proposes that agent architecture stop treating technical acceptance as institutional authorization. Material uncertainty must trigger constraint, escalation, or blocking before reliance.

9. Objections and Limits

9.1 This was a patched bug

The strongest objection is that the paper generalizes from an implementation flaw that providers corrected. If the argument depended on the continued exploitability of the exact envelopes, the objection would prevail. It does not. The incident supplies evidence for a distinction that also applies to memories, summaries, permissions, approvals, and retrieved records. The proposed framework will earn its generality only if it predicts failures in those other artifact classes. Until tested, it remains a theoretical architecture grounded in one unusually revealing case.

9.2 Context binding may be enough

Binding an envelope to user, session, predecessor, and model would close much of the demonstrated attack surface. It would not determine whether the content remains accurate, whether its purpose has changed, or whether new tools make reliance dangerous. Context binding is a major control inside CRAG, not a substitute for the full decision.

9.3 The gate may destroy continuity

Aggressive checks can create false denials, latency, state overhead, and brittle migrations. That is why the proposal includes `CONSTRAIN`, `QUARANTINE`, and audited reissuance. A useful system must preserve benign continuity. The empirical question is not whether maximum restriction reduces attacks. It is whether contextual classification improves the safety-continuity frontier relative to simpler controls.

9.4 Semantic inspection may be impossible

Providers may be unwilling to expose raw reasoning, and exposure itself may create safety and intellectual-property risks. CRAG does not require universal disclosure. It requires that uninspectable content be classified as such and denied roles that depend on semantic verification. A provider may return a sanitized summary, attest to policy properties, perform server-side screening, or keep the trace server-side behind a scoped identifier. What it should not do is let opacity silently count as safety.

9.5 Evolutionary language may overreach

The EPT interpretation is a causal analogy with a defined boundary. The trace exists outside the process that generated it and changes later behavior. The present evidence does not establish evolutionary selection. Claims about replicators, fitness, or stable strategies require a population design and longitudinal measurements. The proposed experiment, not the original attack, would supply that evidence.

9.6 Policy conformance is not production validation

The implemented benchmark evaluates deterministic policy conformance. No claims are made about real-world effect size, latency, resistance to adaptive attacks, superiority over existing memory-security systems, or model compliance. B3's perfect score follows from faithful implementation of the frozen contract. The useful empirical result is comparative and bounded: on these synthetic cases, context binding left same-context semantic and operational failures unresolved, while the full gate classified them without sacrificing the benchmark's benign continuity.

10. Conclusion

Encrypted reasoning traces exposed a fault line that ordinary security language can obscure. The artifacts were authentic enough for provider infrastructure to process. That authenticity did not justify every user, session, model, purpose, or action to which they could be carried.

Agentic systems need to ask four different questions. Is the artifact genuine? Does it belong in this context? What epistemic or instructional role may it play? What consequences may follow if the agent relies on it now? A signature answers only the first.

Contextual runtime admissibility places the decision where the risk materializes: immediately before memory becomes premise, plan, or action. The gate does not solve the problem by deleting history. It distinguishes allowance from constraint, quarantine, escalation, and denial, and it records why the decision was made.

The evolutionary implication is narrower. An opaque trace can outlive its generating process and reorganize later behavior. That makes it a plausible extended phenotype. Whether it becomes a replicator depends on evidence that has not yet been produced: fidelity, descendants, differential survival, and changing frequencies. The policy benchmark does not supply that evidence. A later behavioral population experiment must.

An authentic memory may still be the wrong memory, in the wrong place, with the wrong authority. Systems that can act must learn the difference.

Data and Code Availability

The complete synthetic replication package accompanies this paper in Zenodo at <https://doi.org/10.5281/zenodo.21927142>. The accompanying files are: `cases.jsonl`, the frozen 28-case fixture and its development/holdout designations; `crag_benchmark.py`, the executable B0-B3 policy implementation and benchmark runner; `test_crag_benchmark.py`, the unit and acceptance tests; `method_spec.md`, the hypotheses, metrics, states, and predeclared acceptance criteria; `manifest.json`, the package identity and file map; `source_inventory.json`, the evidentiary-source inventory; `reproduction_matrix.json`, the claim-to-artifact reproduction map; `runs/latest.json`, the machine-readable predictions, metrics, and acceptance result; `comparisons/latest.json`, the B0-B3 comparison record; `provenance/latest.json`, the code, fixture, and result hashes; `report/results.md`, the human-readable report supporting Table 1; `task_ledger.md`, the execution ledger; `status.md`, the completion and archive status; `README.md`, the local reproduction instructions; `CITATION.cff`, the machine-readable citation; and `LICENSE.txt`, the applicable paper, data, documentation, and code licenses. `ZENODO_FILE_MANIFEST.md` reproduces this inventory and the verification role of every deposited file.

The archive does not contain proprietary reasoning traces, reconstructed provider outputs, credentials, personal data, live exploit payloads, or code that queries patched vendor endpoints. All identities, secrets, instructions, and tool consequences represented by the benchmark are synthetic. This boundary permits independent reproduction of the policy-conformance result without creating a replay corpus or a new operational attack channel.

References

- Anthropic. (2026). *Thinking*. Claude Platform Documentation. <https://platform.claude.com/docs/en/about-claude/models/extended-thinking-models>
- Autio, C., Schwartz, R., Dunietz, J., Jain, S., Stanley, M., Tabassi, E., Hall, P., & Roberts, K. (2024). *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*. NIST AI 600-1. <https://doi.org/10.6028/NIST.AI.600-1>
- Google AI for Developers. (2026). *Function calling with the Gemini API: Thought signatures*. <https://ai.google.dev/gemini-api/docs/function-calling>
- Hardy, N. (1988). The confused deputy (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review*, 22(4), 36-38. <https://doi.org/10.1145/54289.871709>
- Lerer, I. A. (2026a). *Runtime Admissibility Before Reliance: Legitimacy Formation and Legitimacy Execution in Legal AI*. Zenodo. <https://doi.org/10.5281/zenodo.20501821>

- Lerer, I. A. (2026b). *Static Permissibility in Dynamic Normative Environments: Dynamic Classification Failure as a Structural Property of Institutional Computation*. Zenodo. <https://doi.org/10.5281/zenodo.21120096>
- Lerer, I. A. (2026c). *The Extended Phenotype of Artificial Intelligence: An Evolutionary Analysis of Model Collapse and the Anthropic Settlement as Cognitive Selection Pressure*. Zenodo. <https://doi.org/10.5281/zenodo.18567734>
- OpenAI. (2026). *Model guidance: Managing conversation state and encrypted reasoning items*. OpenAI API Documentation. <https://developers.openai.com/api/docs/guides/latest-model>
- OWASP Foundation. (2026). *AI Agent Security Cheat Sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/AI_Agent_Security_Cheat_Sheet.html
- Panfilov, A., Schmotz, D., Shumailov, I., Beurer-Kellner, L., Schaeffer, J., Prabhu, A., Geiping, J., & Andriushchenko, M. (2026). *Stealing Reasoning Traces from Proprietary LLM APIs*. arXiv:2608.09867. <https://arxiv.org/abs/2608.09867>
- Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE*, 63(9), 1278-1308. <https://doi.org/10.1109/PROC.1975.9939>
- Zhang, T., Morris, J. X., & Shmatikov, V. (2026). *How to Steal Reasoning Without Reasoning Traces*. arXiv:2603.07267. <https://arxiv.org/abs/2603.07267>

AI-Assistance Disclosure

AI-assisted tools were used to support source organization, structural drafting, language editing, benchmark implementation, testing, and document formatting. The author remains responsible for the final manuscript, claims, citations, code, results, and disclosure.